Anamnésia: Um Gateway de Memória Independente de Agente para Recuperação de Contexto Persistente, Otimização de Tokens e Sistemas de IA Conscientes de Custos

Gabriel (GabezFC)
Projeto Anamnésia (independente)
Relatório técnico v2.1.0 (independente; não é uma publicação IEEE)
Preparado com o agente Hermes, 25 de outubro de 2026
Repositório: `GabezFC/Anamnesia`

Resumo

Agentes de programação e assistentes de conversação pagam repetidamente para reler as mesmas notas, decisões e informações de projetos, e perdem esse conhecimento quando uma sessão termina. Anamnésia é um *gateway de memória* aberto e independente de agente: um serviço local situado entre um agente e um cofre Markdown/Obsidian, que recupera os poucos trechos relevantes para uma consulta com SQLite FTS5/BM25 (opcionalmente enriquecidos por um grafo de conhecimento do Graphify e um avaliador pago de relevância, JEV), os recorta e remove duplicatas dentro de um orçamento de tokens, e retorna um contexto auditável, com sinalização de injeção, por meio do Model Context Protocol (MCP) ou de uma API REST, enquanto registra tokens e custos em um livro de registros. Este artigo (i) documenta a arquitetura implementada a partir do código-fonte, (ii) separa as contribuições originais das funcionalidades fornecidas por componentes externos e (iii) relata um benchmark generativo real, executado anteriormente: 82 perguntas sobre um recorte sanitizado do próprio cofre do autor, nove modelos (oito remotos via OpenRouter, um local via Ollama), cinco pipelines de contexto e 3 797 chamadas de geração registradas. Em comparação com uma referência BM25 de notas completas, o pipeline padrão (recuperação baseline mais o otimizador) reduziu os tokens de entrada informados pelo provedor de 11,103 para 1,776 por consulta e o custo efetivo dos oito modelos remotos em 81.7%, enquanto a acurácia agregada, pontuada por um avaliador LLM, foi estatisticamente indistinguível (86.9% contra 86.6%; $\Delta{=}+0.3$ pp pareado, IC de 95% $[-7.4, +8.0]$). Três resultados negativos ou qualificadores são igualmente importantes: o pipeline híbrido Graphify perdeu 15.0 pp de acurácia; o avaliador pago JEV acrescentou custo sem ganho de acurácia; e uma reanálise estratificada mostra que a paridade do resultado principal esconde uma compensação—nas perguntas em que a referência recuperou um contexto, Anamnésia foi 6.9 pp *menos* acurado, enquanto foi muito melhor nas perguntas em que o mecanismo de abstenção da referência não retornou nada. A persistência da memória entre sessões reais de agentes e a recuperação após conversas longas *não* foram medidas; nós as declaramos não comprovadas. O pacote público do artigo, em `assets/paper-v2.1.0/`, contém os fontes, as figuras e os PDFs; os logs originais e os dados de execução permanecem locais em `data/reports-out`.

Palavras-chave

gateway de memória, geração aumentada por recuperação, Model Context Protocol, BM25, grafo de conhecimento, otimização de tokens, contabilização de custos de LLM, memória de agentes

I. Introdução

Agentes baseados em grandes modelos de linguagem (LLM) não mantêm estado entre sessões e pagam por token dentro delas. Disso decorrem dois problemas práticos. Primeiro, a *continuidade*: decisões, fatos e convenções estabelecidos em sessões anteriores precisam ser fornecidos novamente, geralmente colando notas ou arquivos completos no prompt. Segundo, o *custo*: tokens de entrada dominam a cobrança de cargas de trabalho intensivas em recuperação, e contextos longos também podem degradar a qualidade das respostas [1]. Uma resposta conhecida é a geração aumentada por recuperação (RAG) [2], [3]; sistemas de memória específicos de agentes a estendem com escrita, consolidação e paginação [4]–[7].

Anamnésia (anteriormente *Memory Gateway*) visa a um ponto restrito e prático nesse espaço de projeto: um gateway *local*, *independente de agente* e *auditável* para um cofre Markdown com curadoria humana. Ele não treina modelos e não exige embeddings. A recuperação é lexical (SQLite FTS5 com ordenação BM25 [8], [9]), opcionalmente combinada com um grafo de conhecimento de código/documentos produzido pela ferramenta externa Graphify [10] e com um avaliador pago de relevância/injeção, *JEV*, oferecido pela TypeSafe AI [11]. Os resultados são entregues por MCP [12] ou REST (FastAPI [13]), para que qualquer agente que se comunique por MCP—Hermes, Claude Code, Codex, entre outros—possa usar a mesma memória.

Contribuições. (1) Uma descrição do sistema implementado no nível do código-fonte (Seções III–V), com oito diagramas que refletem os caminhos do código. (2) Um modelo de custos que atribui cada etapa, inclusive o avaliador JEV, ao pipeline que a utiliza (Seção IV-E). (3) Um benchmark generativo documentado, incluindo resultados negativos (Seções VI–VII); os fontes, os dados agregados das figuras, os scripts de visualização e os PDFs integram o pacote público `assets/paper-v2.1.0/`, enquanto os logs originais permanecem locais em `data/reports-out`. (4) Uma separação explícita entre benefícios demonstrados e capacidades ainda não comprovadas (Seções IX, X).

O que este artigo não afirma. Ele não afirma superioridade sobre recuperadores densos, sistemas graph-RAG ou produtos comerciais de memória (nenhum deles foi avaliado), nem afirma que a memória de agentes entre sessões melhora o sucesso em tarefas (isso não foi medido).

II. Fundamentos e Trabalhos Relacionados

RAG e recuperação lexical. RAG condiciona a geração a trechos recuperados [2]; revisões catalogam opções de segmentação, recuperação e reordenação [3]. Recuperadores densos [14] dominam muitos benchmarks de domínio aberto, mas BM25 [9] continua sendo uma referência forte e sem treinamento, especialmente para pequenos corpora privados com identificadores exatos (nomes de testes, flags, números de versão), comuns em notas de engenharia. Métodos de fusão de ordenações, como RRF [15], combinam sinais lexicais e de outros tipos; Anamnésia implementa um modo RRF, mas o desativa por padrão.

Recuperação baseada em grafos. GraphRAG constrói um grafo de entidades e resumos de comunidades para sumarização orientada a consultas [16]; HippoRAG aplica PageRank personalizado sobre um grafo de conhecimento para memória de longo prazo [17], [18]. A ferramenta Graphify usada aqui é de outra natureza: mapeia uma pasta para um grafo com ASTs de tree-sitter para código e extração conduzida por assistente para documentos, e suas arestas são marcadas como extraídas ou inferidas [10]. Conforme medido pelos autores de Anamnésia em 27 de setembro de 2026, o grafo produzido a partir do cofre contém apenas rótulos de páginas e títulos, nunca o texto do corpo (Seção IV-C).

Memória de agentes. MemGPT gerencia uma hierarquia de camadas de memória [4]; Generative Agents armazenam fluxos de experiências e refletem sobre eles [5]; MemoryBank acrescenta curvas de esquecimento [6]; Mem0 extrai e consolida fatos [7]. LoCoMo e LongMemEval medem memória conversacional de longo horizonte [19], [20]. Anamnésia se diferencia por ser independente de agente e baseado em arquivos (as notas continuam sendo Markdown comum), e *não* foi avaliado nesses benchmarks.

Otimização de contexto e custos. Compressores de prompts, como LLMLingua [21], [22], encurtam textos com um modelo de linguagem pequeno; FrugalGPT e RouteLLM reduzem gastos por encadeamento ou roteamento entre modelos [23], [24]. Anamnésia reduz custos *enviando menos contexto*; seus módulos de roteamento e compressão existem, mas são opcionais e não fizeram parte do benchmark. A remoção de quase duplicatas usa ideias de SimHash e MinHash [25]–[27].

LLM como avaliador. Usar um LLM para pontuar respostas livres é comum e apresenta vieses conhecidos [28]; nós o utilizamos, documentamos um defeito de pontuação que ele apresentou e fornecemos uma análise de sensibilidade (Seção VII-C).

III. Arquitetura do Sistema

A Fig. 1 mostra o sistema conforme implementado. Agentes acessam o gateway por MCP sobre stdio (conjunto padrão: a ferramenta única `memory_search`; o conjunto completo tem sete ferramentas, com `memory_save` e `memory_feedback` habilitadas opcionalmente), por REST (cerca de 130 operações, com escritas protegidas

por verificação de loopback, correspondência de Origin/Referer e um `X-MG-Token`), ou por uma ferramenta de linha de comando (`anamnesia`: `doctor`, `start`, `mcp`). Uma ponte opcional MCP-para-REST encaminha apenas `memory_search`. Todos os caminhos entram no mesmo objeto em processo, `MemoryGateway`, cuja implementação de busca executa as etapas detalhadas na Seção IV.

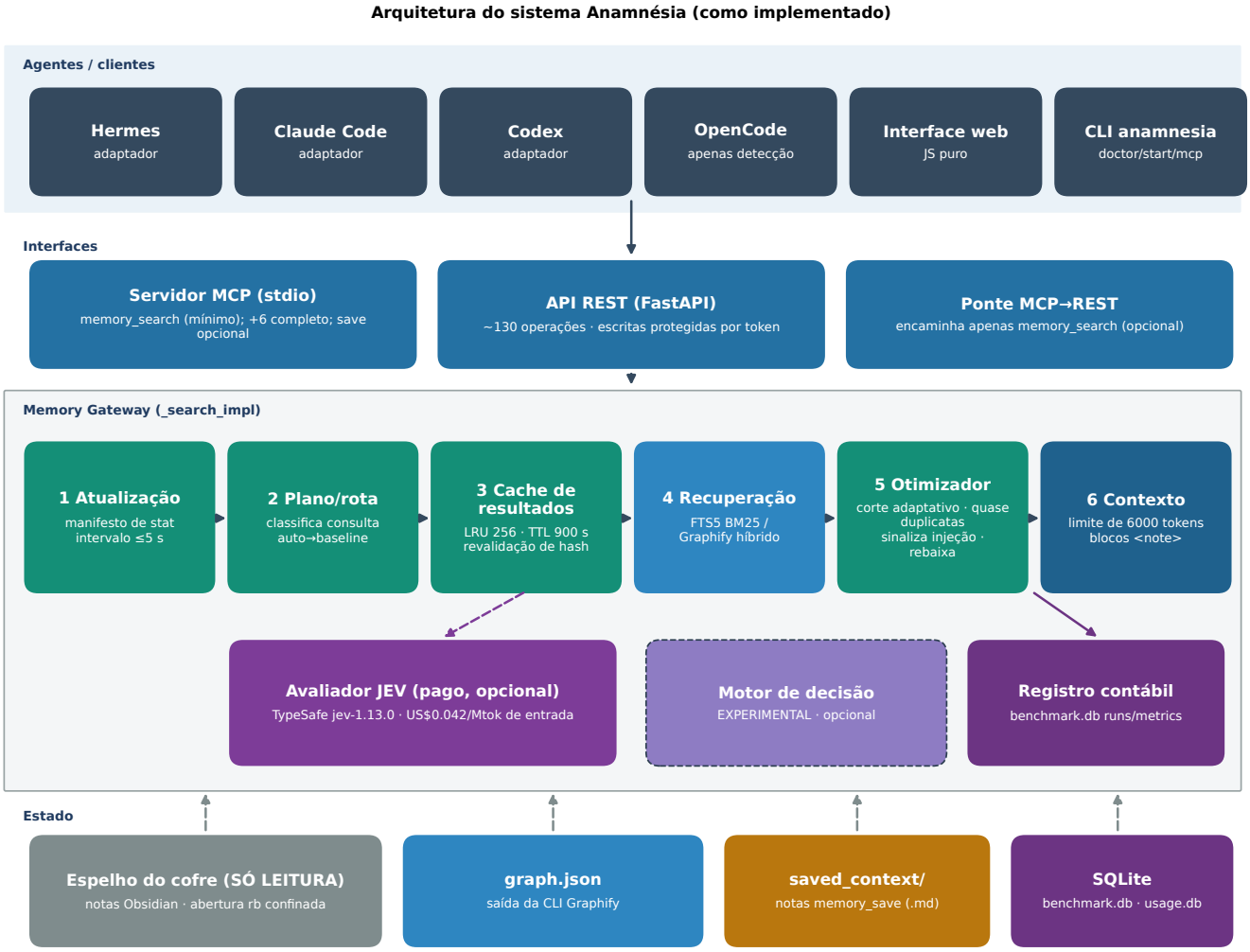


Figura 1. Arquitetura do sistema conforme implementada. Caixas tracejadas são opcionais ou experimentais. O espelho do cofre é somente leitura; notas escritas por agentes ficam em uma pasta separada `saved_context/`.

Estado. O cofre do usuário é espelhado byte a byte em um diretório de dados e aberto somente para leitura por chamadas confinadas de `open(rb)`; fluxos de dados alternativos do NTFS e nomes reservados de dispositivos são rejeitados. Notas escritas por agentes são armazenadas separadamente em `saved_context/` (Seção V). Um arquivo SQLite (`benchmark.db`) mantém o *livro de registros* (`runs`, `candidates`, `budgets`, `routing_decisions`, `verification_results`, `jev_cache` e outros); um segundo arquivo mantém estatísticas de uso. Os campos de tokens e custos não são colunas, mas chaves de um blob JSON (`metrics_json`: `jev_input_tokens`, `jev_cost`, `context_tokens`, `total_tokens_spent`, ...).

Componentes e procedência. A Tabela I separa o que Anamnésia implementa do que utiliza.

A lista de dependências Python do repositório é curta (FastAPI, uvicorn, `mcp`, `httpx`, Pydantic [31], `typesafe-sdk`, PyYAML, `anthropic`); em particular, *não* há pacote BM25 separado nem modelo de embeddings, e `tiktoken` [32] não é uma dependência de execução (foi usado somente pelo aparato de benchmark para uma contagem independente de tokens).

Tabela I
PAPEL E PROCEDÊNCIA DOS COMPONENTES. "ORIGINAL" SIGNIFICA IMPLEMENTADO NO REPOSITÓRIO DE ANAMNÉSIA.

Componente	Papel em Anamnésia	Procedência
SQLite FTS5	índice lexical, ordenação `bm25()` [8]	externo
Graphify CLI	constrói `graph.json` a partir do espelho do cofre [10]	externo (Apache-2.0)
Recuperação híbrida	sementes BM25 + expansão BFS no grafo	original
Otimizador	corte, quase duplicatas, sinalização de injeção, cache, orçamento	original
JEV (TypeSafe)	avaliador pago de relevância/injeção [11]	serviço externo; lógica cliente original
Servidor MCP	interface de ferramentas para agentes [12]	protocolo externo; servidor original
FastAPI/uvicorn	camada REST [13]	externo
Ollama	ambiente de execução de modelo local no benchmark [29]	externo
Registros/modelo de custos	contabilização de tokens e custos	original
Laya	modelo local opcional de relevância [30]	externo, opcional
Decision Engine	estrutura de manutenção/descarte independente de provedor	original, *experimental*

IV. RECUPERAÇÃO E OTIMIZAÇÃO DE MEMÓRIA

A. Caminho da consulta

A Fig. 2 resume as etapas de uma busca; sua ordem é fixa no gateway: (1) uma verificação de atualização que compara um manifesto de metadados stat do cofre no máximo a cada `MG_VAULT_REFRESH_S` $= 5\,\mathrm{s}$ e reconstrói o índice quando há mudanças (limpando os caches); (2) classificação da consulta em SIMPLE/MEDIUM/COMPLEX/AMBIGUOUS e roteamento (todas as classes usam o pipeline `baseline` por padrão, portanto `pipeline=auto` resolve para baseline, a menos que a política opcional *economy* selecione um pipeline JEV); (3) proteção opcional de orçamento; (4) consulta ao cache de resultados; (5) pipeline de recuperação; (6) nova verificação de escopo; (7) pós-filtro; (8) construção do contexto; (9) gravação de métricas/registros e inserção no cache.



Figura 2. Fluxo da consulta à resposta. JEV participa apenas dos pipelines `graphify_jev*`.

B. Recuperação lexical

As notas são divididas em seções (título mais corpo) e indexadas em uma tabela virtual FTS5 com o tokenizador `unicode61 remove_diacritics 2`. A ordenação usa a função nativa `bm25(sec,0,w,1,0,0)`, com peso

de título $w = 2.0$ e peso de corpo 1. O `bm25()` do SQLite implementa a forma Okapi [8], [9]

$$s(d,q) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \tag{1}$$

onde SQLite fixa k_1=1.2 e b=0.75; o código de Anamnésia não os sobrescreve (esta é nossa inferência a partir do código, não uma declaração explícita nele). SQLite retorna pontuações com sinal invertido, que Anamnésia reverte. Até `MAX_CANDIDATES` = 100 seções são mantidas por consulta.

C. Recuperação híbrida Graphify

A CLI externa Graphify (`graphify update`, somente AST neste uso) transforma o espelho do cofre em `graph.json`. Uma nota de projeto no repositório registra que, no cofre do autor, esse grafo continha 1,109 nós (74 páginas e 1,035 títulos) e *nenhum texto de corpo*, de modo que iniciar apenas pelos rótulos do grafo recuperou 8 de 10 perguntas de desenvolvimento e falhou quando a resposta estava apenas no corpo de uma nota. Por isso, Anamnésia implementa sua própria busca *híbrida* (Fig. 3): os primeiros dez arquivos distintos de BM25 são sementes textuais; a expansão em largura no grafo acrescenta até 15 arquivos relacionados. Candidatos-semente recebem a pontuação em escada de posições $1.0 - 0.45\,r/n_{\text{seeds}}$ (faixa $1.0 \to \approx 0.55$), e candidatos apenas do grafo recebem $0.5 - 0.2\,i/n_{\text{rel}}$ ($0.5 \to 0.3$). A expansão por PageRank personalizado (α=0.85, 30 iterações, top-15) existe, mas fica desativada por padrão. Essas pontuações são *posições, não probabilidades de relevância*, um fato com consequências na Seção VII.

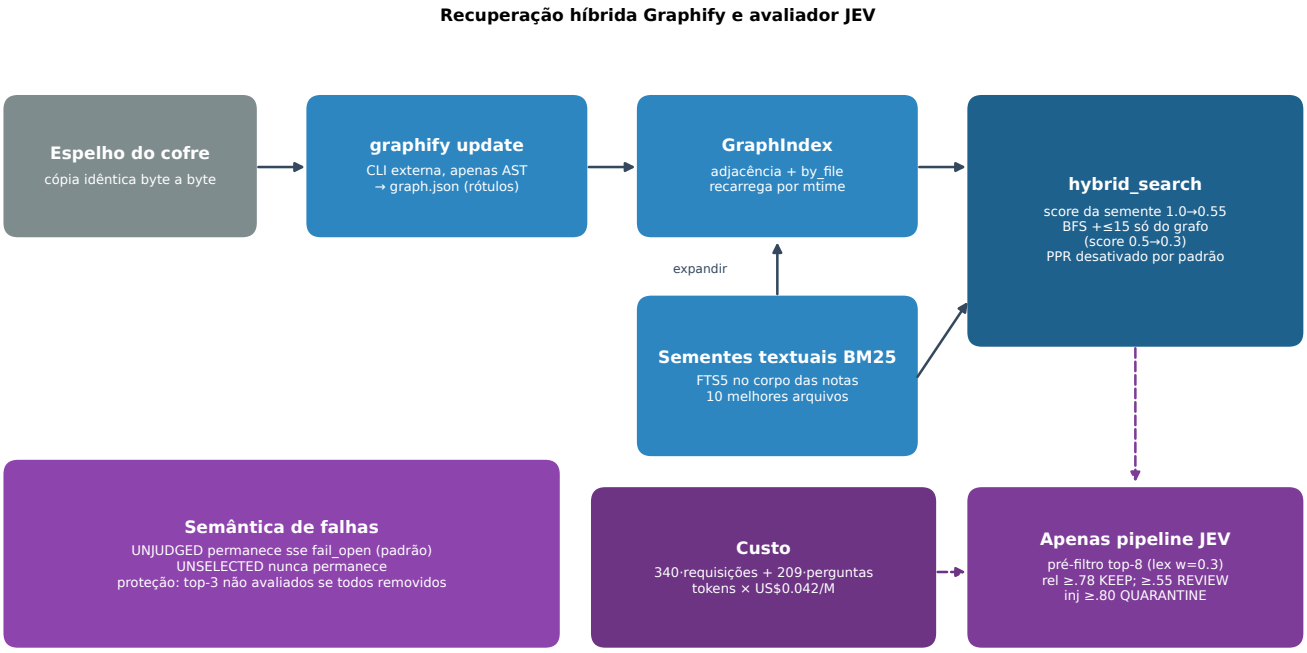


Figura 3. Recuperação híbrida Graphify e avaliador JEV. O grafo fornece estrutura; BM25 fornece pontos de entrada endereçáveis por conteúdo.

D. Camada de Otimização de Memória

Após a recuperação, cada trecho candidato tem seus espaços em branco normalizados, é truncado a `SNIPPET_MAX_TOKENS` = 300 e recebe um hash. O pós-filtro (Fig. 4) aplica, nesta ordem:

1) **Corte adaptativo** (somente pipelines gratuitos; consultas SIMPLE/MEDIUM): manter o candidato i se $i < 3$ ou $\text{score}_i \geq 0.5 \cdot \max_j \text{score}_j$. Como as pontuações BM25 de baseline não têm limite superior, enquanto as do Graphify formam uma escada de posições, a mesma razão atua de modo diferente nos dois pipelines.
2) **Colapso de quase duplicatas**: SimHash de 64 bits com 8 bandas LSH (Hamming ≤ 12) e um pré-filtro Jaccard de 4-shingles selecionam pares de candidatos, confirmados por uma razão de similaridade de sequências ≥ 0.88 [25]–[27]. Uma seção por nota é mantida por padrão (`DEDUP_MAX_PER_NOTE` $= 1$).
3) **Sinalização de injeção**: uma triagem por expressões regulares ("inj-screen-v2", inglês e português, normalizada em Unicode) marca notas suspeitas com um atributo de aviso. Ela *sinaliza, mas nunca descarta*, e resultados sinalizados não são armazenados em cache.
4) **Rebaixamento temporal e por conflito**: notas históricas ou conflitantes têm sua pontuação multiplicada por 0.5.

O *construtor de contexto* então ordena os candidatos, intercala-os por arquivo em round-robin, limita cada fonte a 1,500 tokens e preenche de forma gulosa um orçamento total de 6,000 tokens, descartando um bloco se restassem menos de 120 tokens. Cada bloco é envolvido em um elemento `<note source=...>`, e todo o contexto é declarado não confiável.

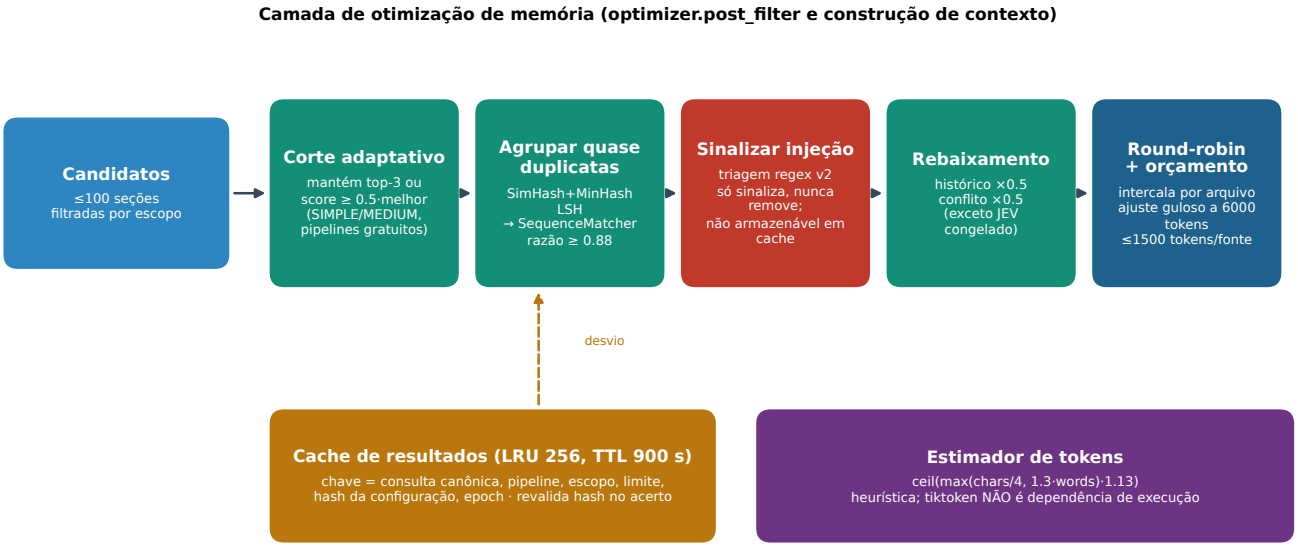


Figura 4. Camada de Otimização de Memória: operações de pós-filtro, cache de resultados e estimador de tokens.

Cache. O cache de resultados é um LRU em processo (256 entradas, TTL de 900 s), cuja chave reúne consulta canônica, pipeline, escopo, orçamento, resumo criptográfico da configuração e época do índice; a cada acerto, novos hashes dos arquivos entregues são calculados para detectar desatualização, e regras contra envenenamento recusam armazenar resultados sinalizados. *Não* encontramos cache semântico baseado em embeddings para resultados de busca; o único cache com consultas normalizadas é o de vereditos do avaliador (termos da consulta mais hash do conteúdo da nota).

Estimativa de tokens. Sem uma dependência de tokenizador, Anamnésia estima $\hat{n} = \lceil \max(c/4,\ 1.3w) \cdot 1.13 \rceil$ a partir das contagens de caracteres e palavras, onde 1.13 é um fator de calibração para português que o próprio código descreve como não medido recentemente. Todas as contagens de tokens dos nossos resultados vêm do *provedor* (`usage.prompt_tokens`) ou de `tiktoken` (tamanho do contexto), não desse estimador.

E. Avaliador JEV e modelo de custos

Nos pipelines `graphify_jev*`, um pré-filtro lexical mantém os 8 primeiros candidatos, que JEV (modelo `jev-1.13.0`) pontua quanto à relevância e à injeção. Um candidato recebe `QUARANTINE` se injeção ≥ 0.80, `KEEP` se relevância ≥ 0.78, `REVIEW` se ≥ 0.55, ou é descartado nos demais casos; falhas do avaliador seguem `fail_open` por padrão (candidatos não avaliados sobrevivem), e, se tudo for descartado, os 3 primeiros candidatos não avaliados são restaurados como rede de segurança. O ajuste por mínimos quadrados do repositório sobre dez requisições reais fornece

$$\begin{aligned} \hat{n}_{\text{JEV}} &\approx 340\,R + 209\,Q \quad \text{(tokens de entrada)}, \\ C_{\text{JEV}} &= \frac{n_{\text{JEV}}}{10^6} \cdot 0.042 \text{ US\$}, \end{aligned} \tag{2}$$

para R requisições HTTP e Q perguntas avaliadas; tokens de saída são gratuitos na tarifa citada. O custo efetivo de um pipeline que atende a um modelo m é, portanto,

$$\begin{aligned} C_m^{\text{with}} &= \sum_q \Big[C_{\text{gen}}\big(m, \text{ctx}_q\big) + C_{\text{JEV}}(q) \Big], \\ C_m^{\text{without}} &= \sum_q C_{\text{gen}}\big(m, \text{notes}_q\big), \end{aligned} \tag{3}$$

com recuperação e otimização locais custando zero dólares (apenas CPU; energia não medida). Uma economia é $1 - C^{\text{with}}/C^{\text{without}}$. A redução de contexto, isoladamente, *não* é economia a menos que a Eq. (3) a confirme; a Fig. 5 mostra o que é cobrado. O Decision Engine experimental encapsula JEV e outros provedores em uma interface única com cache e alternativa por operação; ele não foi usado em nenhum experimento relatado aqui, e sua própria telemetria identifica os dólares de JEV como desconhecidos.

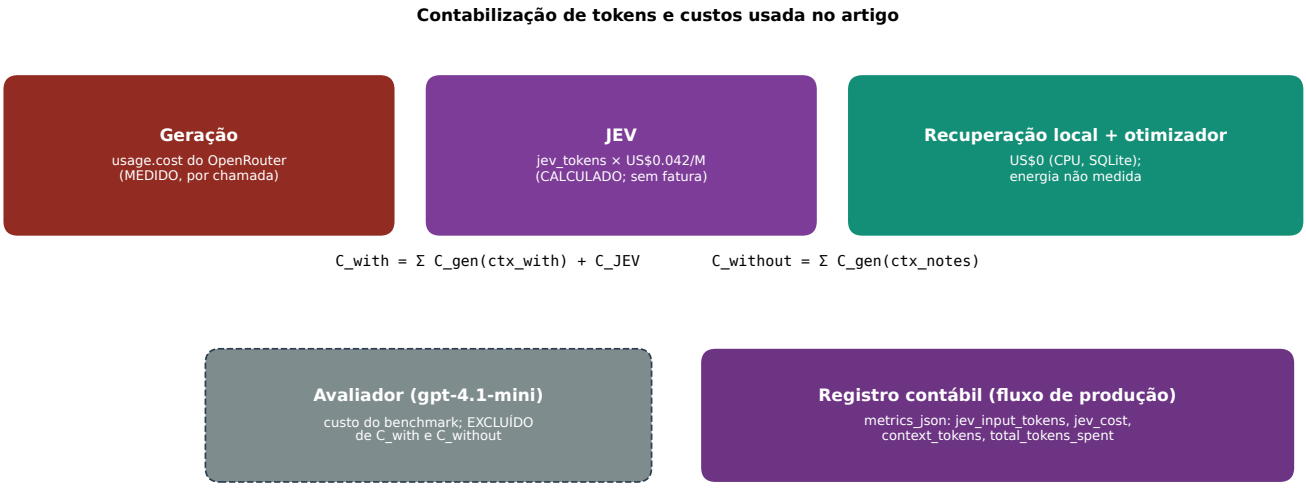


Figura 5. Contabilização de custos usada em todo o artigo. O avaliador da avaliação é uma despesa adicional do experimento e é excluído de ambos os braços.

V. Memória Persistente e Integração com Agentes

Modelo de armazenamento. Com `MG_MCP_SAVE=1`, `memory_save` grava uma nota Markdown com frontmatter YAML (projeto, confiança, confiabilidade e contadores de feedback) em `saved_context/`, nunca no cofre do usuário. As escritas são atômicas (arquivo temporário, `fsync`, `os.replace`) sob um bloqueio de arquivo entre processos (`msvcrt.locking` no Windows, `fcntl.flock` nos demais sistemas). Gravações passam por uma validação de projeto, limites de tamanho e frequência, e um detector de segredos que rejeita textos

semelhantes a segredos. A pasta é montada como uma raiz extra somente leitura do índice, de modo que uma atualização (Fig. 6) torna uma nota salva recuperável por buscas posteriores, inclusive em outros processos e sessões. `memory_feedback` ajusta a confiança de uma nota em $+0.05$ (útil), -0.2 (errada) ou -0.05 (desatualizada); uma ordenação composta ("v2") que a utiliza é opcional.

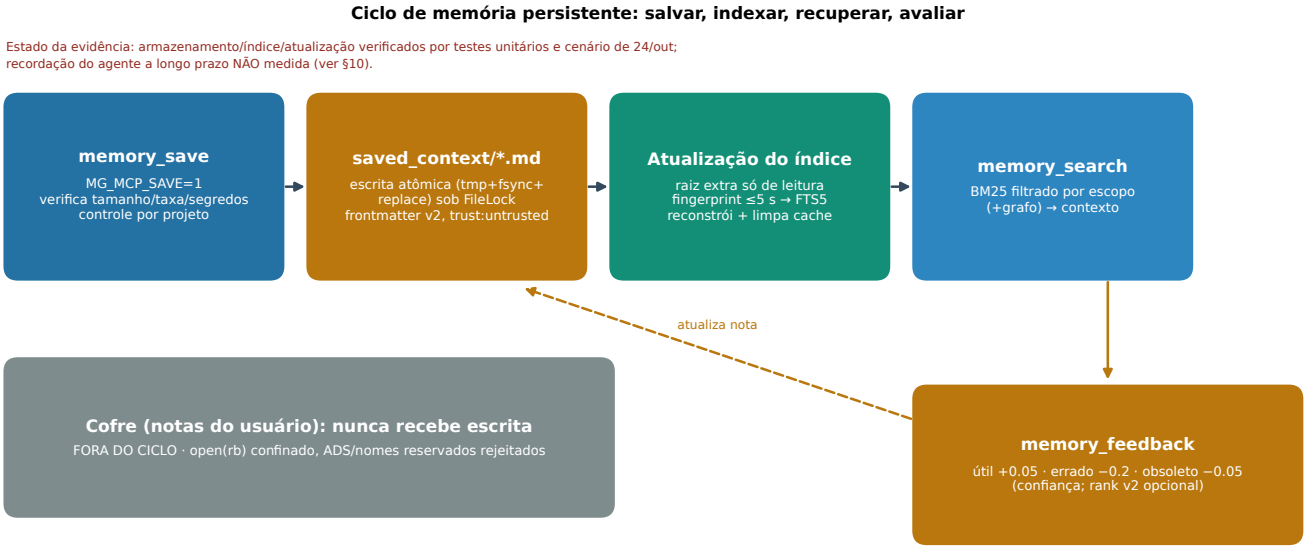


Figura 6. Ciclo de memória persistente. O mecanismo está implementado; seu benefício de ponta a ponta para agentes não foi medido.

Isolamento. Buscas aceitam um escopo como `projeto:X`; `MG_SCOPE` pode fixar uma sessão de modo que ferramentas possam apenas *restringi-lo*. Por padrão, o escopo é aplicado após o corte dos 100 primeiros candidatos, portanto um escopo restrito pode retornar menos resultados do que os existentes naquele projeto. Uma nova verificação de defesa em profundidade conta e bloqueia resultados vazados.

Integração com agentes. A Fig. 7 mostra os caminhos de comunicação. Existem adaptadores para Hermes, Claude Code e Codex; OpenCode tem apenas detecção e não foi validado; não encontramos integração com Cursor. Modelos locais e remotos são acessados por um cliente genérico compatível com OpenAI (adaptadores Ollama, vLLM, OpenRouter, OpenAI, Anthropic; vLLM não testado contra um servidor real). Leituras na vinculação loopback padrão não precisam de token; rotas de alteração exigem cliente loopback, Origin e Referer correspondentes, e `X-MG-Token`. Notas recuperadas carregam `trust="untrusted"`. O painel no navegador usa módulos ES puros com um roteador por hash e sem etapa de compilação.

Experimental ou inacabado. As predefinições do Decision Engine, Colibrì (backend JEV local), compressores e reordenadores LLMLingua/Provence (ambiente Python separado) e o catálogo de notas SQLite não estão conectados à execução padrão; Laya executa em CPU, mas é opcional e, segundo as próprias medições do repositório, é um modelo fraco de relevância (AUC 0.64–0.68 contra 0.975 para sobreposição lexical).

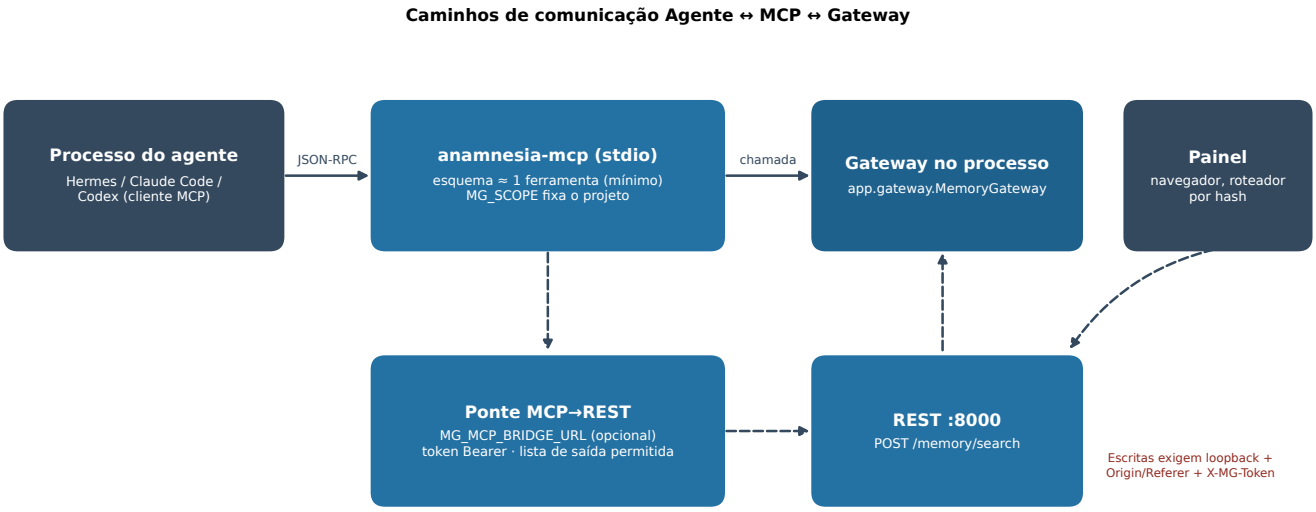


Figura 7. Comunicação entre agentes, servidor MCP e gateway.

VI. Metodologia Experimental

Escopo das evidências. Todos os resultados abaixo foram produzidos *antes* deste artigo por duas execuções anteriores: um benchmark apenas de recuperação (24 de outubro de 2026) e um benchmark generativo (25 de outubro de 2026). Nenhum experimento foi executado para o artigo; tabelas e figuras são regeneradas a partir dos logs armazenados. Quando calculamos novas estatísticas a partir dos dados armazenados (bootstraps pareados, estratificação, uma regra de sensibilidade), isso é declarado, e os vereditos originais permanecem inalterados.

Corpus e perguntas. *D2* é um instantâneo sanitizado das notas reais de projetos do autor (português, 379 notas), congelado em uma cópia; o cofre original nunca recebeu escritas. As 82 perguntas têm fontes de referência verificáveis e respostas em expressões regulares: 36 factuais, 28 numéricas, 10 sem resposta, 3 ambíguas, 3 de duplicatas/versões, 2 de documentos longos. (Existe um corpus sintético reproduzível, D1, mas ele não foi usado generativamente.)

Braços (Fig. 8). *Sem*: um aparato independente (importações de Anamnésia são bloqueadas por uma proteção) com BM25 sobre notas completas, entregando as 5 primeiras notas; seu limiar de abstenção θ é calibrado em uma partição de desenvolvimento, e ele retornou contexto vazio para 22 de 82 perguntas (10 sem resposta, 12 respondíveis). *Com*: o Anamnésia real via MCP stdio, com diretório pessoal novo e cópia do cofre por bloco, offline: **baseline** (seções BM25), **Graphify** (híbrido), **auto+otimizador** (`pipeline=auto`, que resolve para baseline, mais o otimizador e o cache) e **Graphify+JEV** (pago). Contextos foram produzidos uma vez (passagem fria) e reapresentados *de forma idêntica byte a byte* a cada modelo, portanto o único fator variável entre os braços é o contexto.

Geração. Um prompt de sistema fixo instrui o modelo a responder apenas com base no contexto, citar fontes e, caso contrário, responder "Não encontrei essa informação no contexto". Temperatura 0, seed 42, 512 tokens de saída, uma repetição por célula. Oito modelos via OpenRouter [33] (gpt-4o-mini, gpt-4.1-mini, gemini-2.5-flash-lite, deepseek-chat-v3.1, llama-3.3-70b-instruct, qwen3-235b-a22b-2507, mistral-small-3.2-24b-instruct, claude-haiku-4.5) e um modelo local (qwen3.5:9b via Ollama em uma RTX 3060 compartilhada com outros processos). O custo remoto é o `usage.cost` informado pelo provedor; o custo JEV é *calculado* (Eq. 2); nenhuma fatura da TypeSafe estava disponível.

Pontuação. Um avaliador LLM independente (gpt-4.1-mini, temperatura 0) comparou cada resposta com o trecho de referência e retornou CORRECT, INCORRECT ou ABSTAIN (para perguntas sem resposta: ABSTAIN é correto). A acurácia é a proporção de CORRECT nas 72 perguntas respondíveis. Intervalos para proporções individuais são

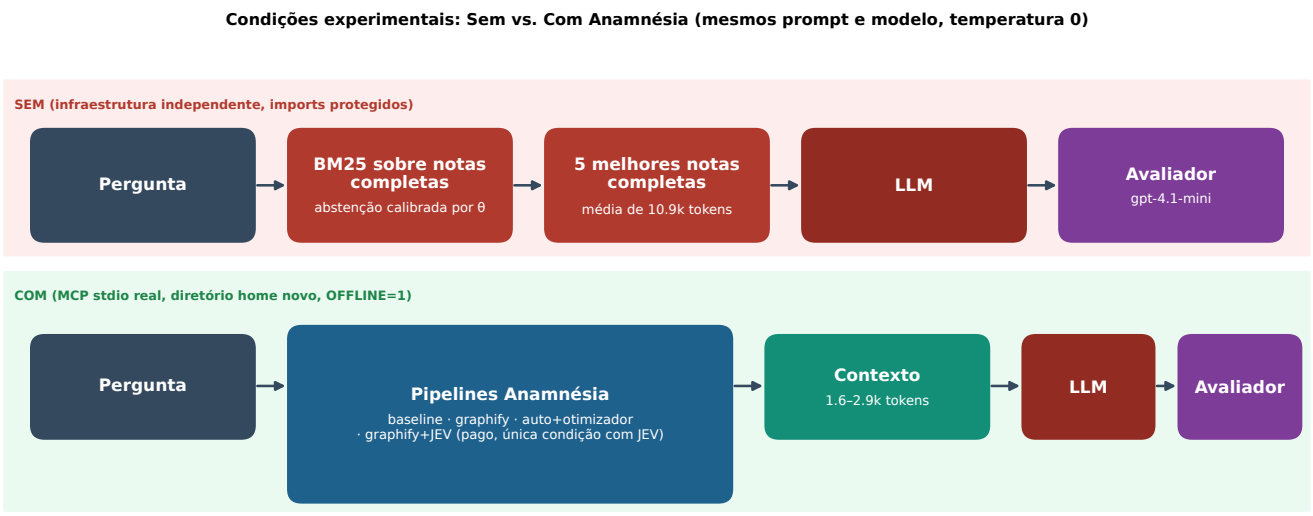


Figura 8. Desenho experimental: prompt, modelo e temperatura idênticos; apenas o contexto difere.

intervalos de Wilson [34]; diferenças pareadas usam um bootstrap no nível da pergunta [35] (4,000 reamostragens, seed 20261025), agregando os nove modelos e tratando perguntas como unidade amostral.

Volume. Foram registradas 3,797 linhas de geração, 3,690 válidas (9 modelos $\times$ 5 braços $\times$ 82 consultas) e 107 tentativas malsucedidas (HTTP 429/400), repetidas até cada célula ter 82 respostas; foram produzidos 3,690 julgamentos. O custo de geração foi US$4.60, e o de avaliação US$0.75; o saldo do OpenRouter caiu cerca de US$5.38.

Benchmark apenas de recuperação (24 de outubro). Conjuntos D1 (sintético), D2 (real) e D3 (D2 antes/depois de uma alteração na estrutura do cofre), 5 repetições medidas após um aquecimento descartado, passagens frias e quentes, tiktoken `cl100k_base` para todas as contagens de tokens, bootstrap pareado com 2,000 reamostragens (seed 20261024). Ele também inclui uma referência *OFF-B* mais forte: BM25 sobre *seções* com o mesmo orçamento de tokens da execução correspondente de Anamnésia.

VII. Resultados Experimentais

A. *Tokens, custo e acurácia*

A Tabela II e a Fig. 9 apresentam os resultados agregados. Os tokens de entrada informados pelo provedor caem de 11,103 (Sem) para 1,776 (auto+otimizador), uma redução de 84%; o tamanho médio do contexto entregue, medido com tiktoken, cai de 10,889 (máximo 71,308) para 1,637. O comprimento médio de saída é de cerca de 60–70 tokens em todos os braços, portanto o custo é quase inteiramente de entrada. O custo efetivo dos oito modelos remotos cai 81.7% para auto+otimizador (80–86% por modelo, Fig. 12); JEV acrescenta US$0.0101 por execução de 82 consultas (252,404 tokens do avaliador), mas, como o braço JEV também entrega contextos maiores, sua economia agregada é de apenas 68.8% (59–73% por modelo).

A acurácia é estatisticamente indistinguível da referência para auto+otimizador ($+0.3$ pp; Tabela III), assim como para o pipeline baseline simples ($+2.2$ pp) e para Graphify+JEV (-1.5 pp). Graphify sem JEV é o único braço com perda significativa (-15.0 pp, IC excluindo zero). Quatro de nove modelos pontuam pelo menos tão bem com auto+otimizador quanto sem ele. A acurácia em uma única célula tem IC de 95% de aproximadamente ± 8 pp (Fig. 10); apenas comparações agregadas são informativas.

As economias acompanham o preço do modelo: para claude-haiku-4.5, o custo de 82 consultas cai de US$1.10 para US$0.22; para mistral-small-3.2, de US$0.072 para US$0.012, portanto a economia absoluta é pequena para modelos baratos. Para o modelo local não há economia em dólares (custo de API zero em todos os braços), embora sua latência mediana de geração tenha caído de 8.1 s para 2.6 s com auto+otimizador; essa comparação é confundida pelo compartilhamento de GPU e pelo carregamento do modelo no Ollama (a primeira chamada levou 6.8 s). A

Tabela II

RESULTADOS AGREGADOS DE NOVE MODELOS (72 PERGUNTAS RESPONDÍVEIS CADA; ACURÁCIA n=648 POR BRAÇO). O CUSTO SOMA OS OITO MODELOS REMOTOS (US$ POR 82 CONSULTAS POR MODELO); O CUSTO JEV É CALCULADO E, PARA COMPARABILIDADE, ATRIBUÍDO A CADA MODELO (O CONTEXTO DO AVALIADOR FOI PRODUZIDO UMA VEZ; O GASTO REAL COM JEV FOI DE CERCA DE US$0.010); O CUSTO DE GERAÇÃO É MEDIDO. AS ECONOMIAS SÃO RELATIVAS AO BRAÇO *Sem*.

Braço	Tok. entrada	Acur. (%)	Custo	dos quais JEV	Economia	US$/correta$\times 10^3$
Sem (BM25, notas completas)	11,103	86.6	2.340	0.0000	–	4.69
Com: baseline	2,688	88.7	0.608	0.0000	74.0%	1.19
Com: Graphify híbrido	2,460	71.6	0.571	0.0000	75.6%	1.38
Com: auto + otimizador	1,776	86.9	0.428	0.0000	81.7%	0.86
Com: Graphify + JEV	2,908	85.0	0.731	0.0807	68.8%	1.50

Tabela III

DIFERENÇA PAREADA DE ACURÁCIA EM RELAÇÃO A *Sem* (PONTOS PERCENTUAIS; BOOTSTRAP NO NÍVEL DA PERGUNTA SOBRE AS 72 PERGUNTAS RESPONDÍVEIS, 4000 REAMOSTRAGENS, SEED 20261025, NOVE MODELOS AGREGADOS).

Braço	Δ acur. (pp)	IC de 95%
Com: baseline	+2.2	[-4.9, +9.4]
Com: Graphify híbrido	-15.0	[-25.5, -4.2]
Com: auto + otimizador	+0.3	[-7.4, +8.0]
Com: Graphify + JEV	-1.5	[-10.6, +7.9]

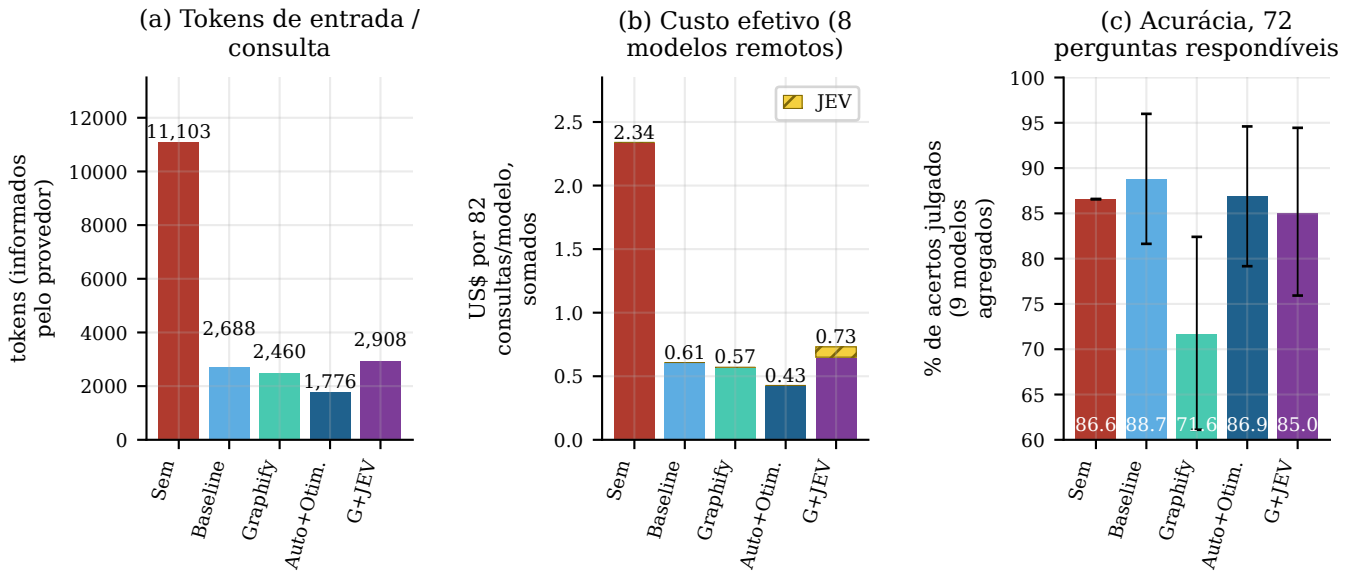


Figura 9. Resultados principais agregados. (a) Tokens de entrada por consulta informados pelo provedor. (b) Custo efetivo dos oito modelos remotos; hachurado = JEV (calculado). (c) Acurácia nas 72 perguntas respondíveis; barras de erro são ICs bootstrap de 95% da diferença pareada em relação a *Sem*, desenhados em torno do valor de cada braço.

Tabela IV

RESULTADOS POR MODELO (EXECUÇÃO ÚNICA, TEMPERATURA 0, 82 CONSULTAS). CUSTO = CUSTO MEDIDO DE GERAÇÃO (+ JEV CALCULADO NA ÚLTIMA COLUNA) EM US$; "ECONOMIA" É A ECONOMIA DE CUSTO EFETIVO EM RELAÇÃO A *Sem*; ACURÁCIA SOBRE 72 PERGUNTAS RESPONDÍVEIS. O MODELO LOCAL TEM CUSTO DE API ZERO E É EXCLUÍDO DAS COLUNAS DE ECONOMIA.

	Sem		Baseline		Graphify		Auto+otim.		Graphify+JEV		
Modelo	Custo	Acur.	Econ.%	Acur.	Econ.%	Acur.	Econ.%	Acur.	Custo	Econ.%	Acur.
local:qwen3.5:9b	0	86	–	90	–	69	–	92	0.010	–	86
mistral-small-3.2-24b-instruct	0.072	88	74	88	76	74	83	85	0.030	59	85
gemini-2.5-flash-lite	0.096	88	74	92	77	71	82	83	0.036	62	83
qwen3-235b-a22b-2507	0.111	88	75	89	76	75	83	86	0.040	64	86
gpt-4o-mini	0.128	88	74	83	77	68	82	81	0.046	65	83
llama-3.3-70b-instruct	0.148	88	74	86	75	72	82	86	0.048	67	82
deepseek-chat-v3.1	0.340	83	79	93	78	74	86	88	0.090	73	89
gpt-4.1-mini	0.344	88	74	88	76	69	82	89	0.107	69	88
claude-haiku-4.5	1.102	85	72	90	74	72	80	93	0.334	70	83

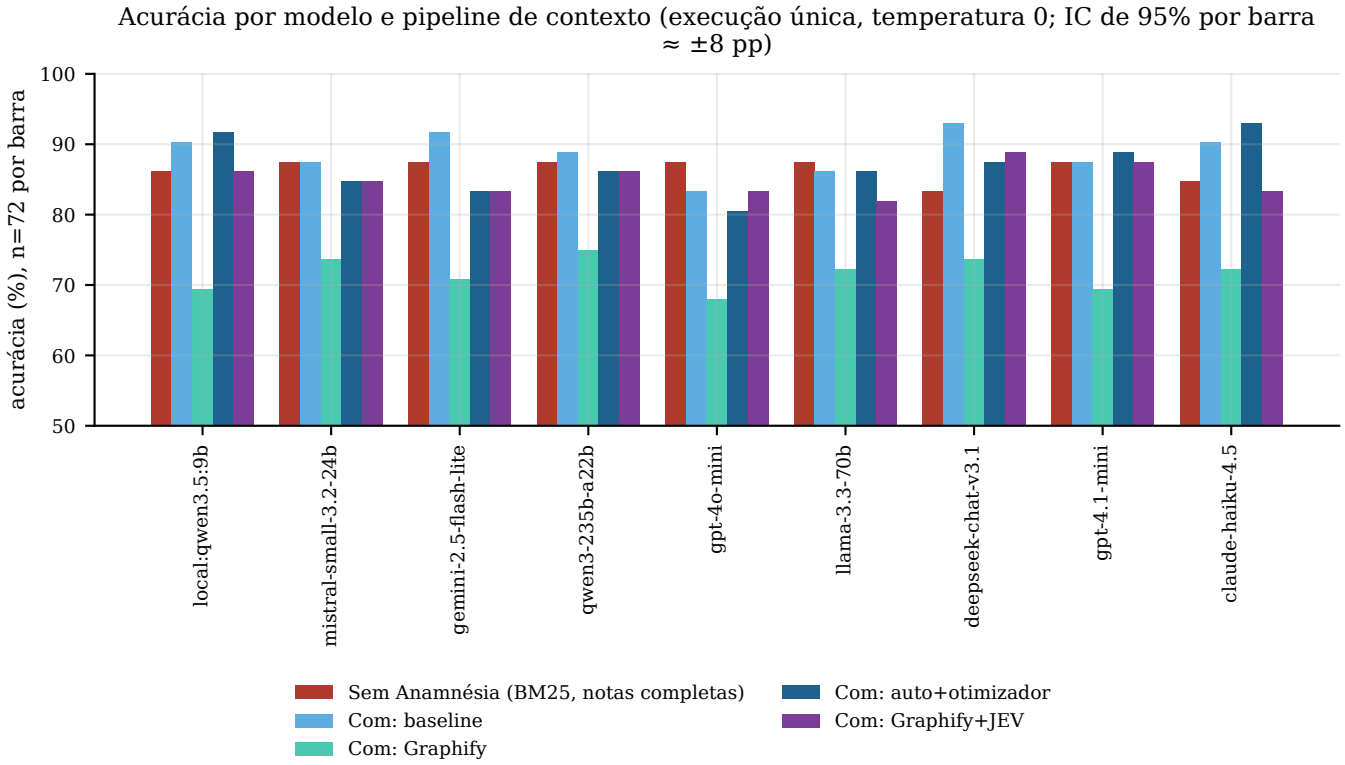


Figura 10. Acurácia por modelo e pipeline (72 perguntas por barra, execução única).

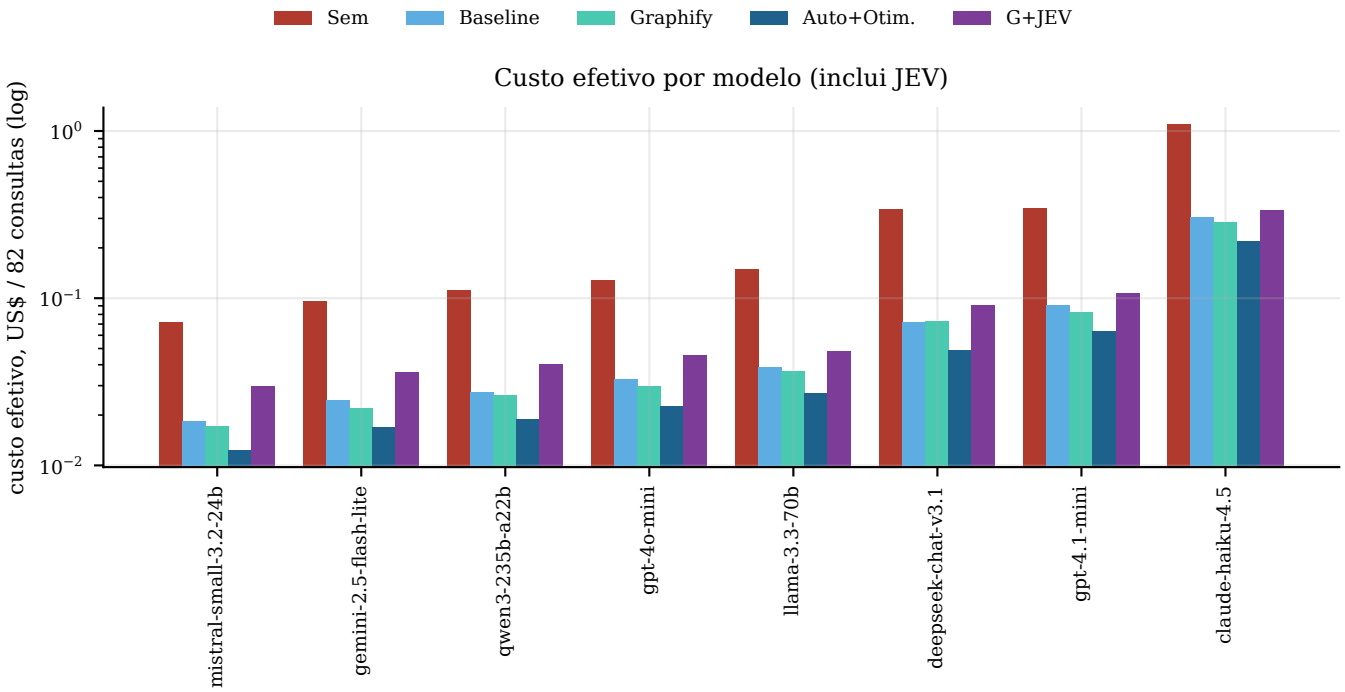


Figura 11. Custo efetivo por modelo (escala logarítmica), incluindo JEV.

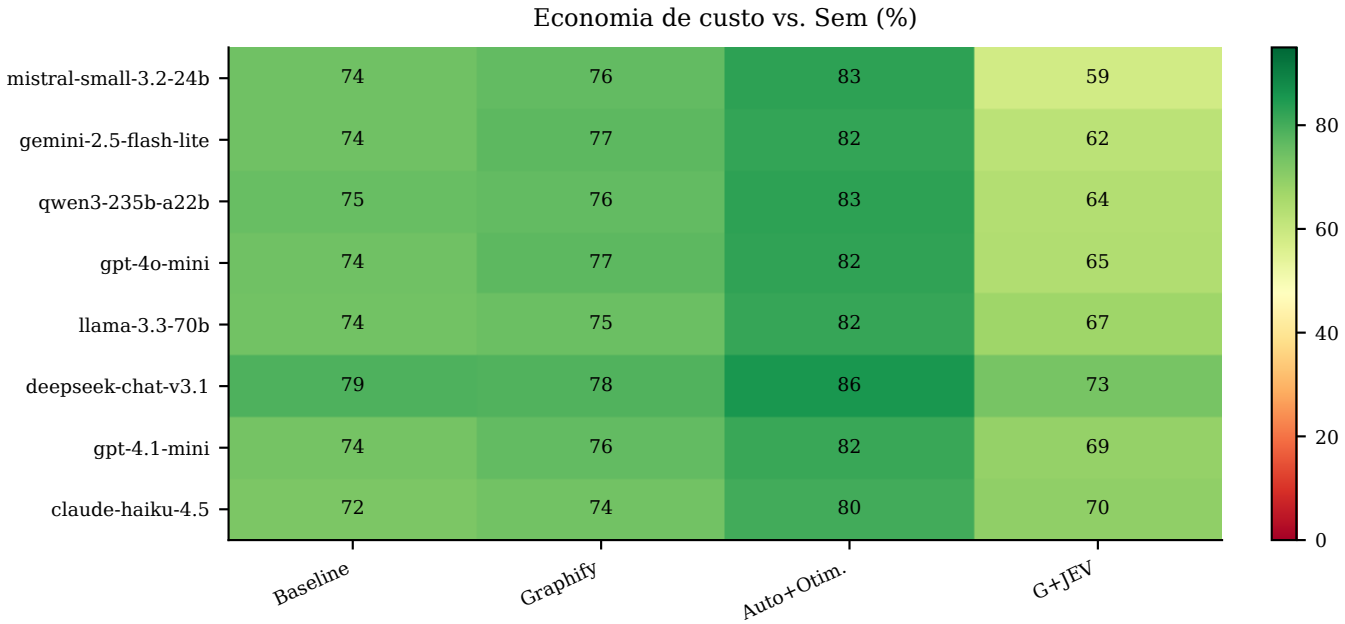


Figura 12. Economia de custo efetivo em relação a *Sem*, por modelo e pipeline (%).

latência de geração remota mudou menos (média dos p50 por modelo: 2.98 s Sem contra 2.30 s auto+otimizador) e é dominada pelas filas do provedor com seis chamadas simultâneas. A Fig. 13 apresenta custo em função da acurácia.

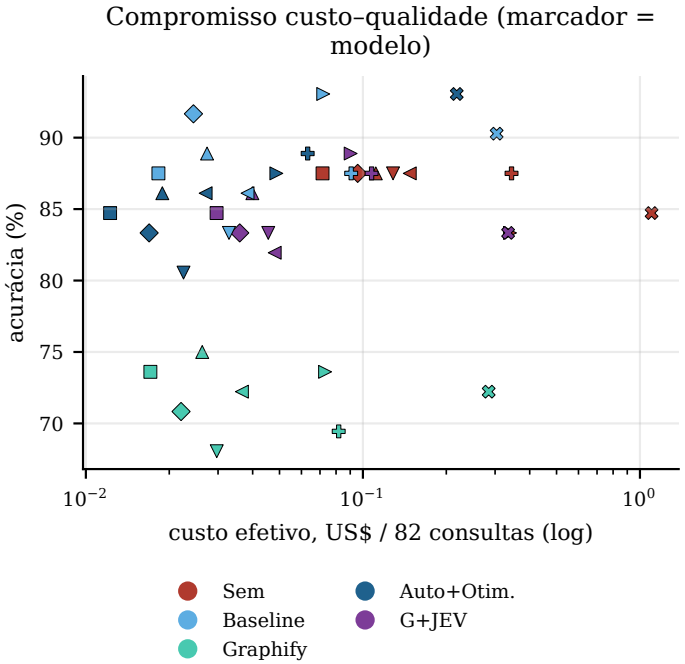


Figura 13. Compensação entre custo e qualidade. O formato do marcador identifica o modelo.

B. Qualidade por classe de pergunta

A Tabela V e a Fig. 14 mostram que a perda de Graphify se concentra nas perguntas numéricas (57% contra 94% Sem), nas quais o número exato está em corpos de notas que o grafo apenas de rótulos não indexa. A resposta de referência estava presente no contexto entregue em 53% das perguntas respondíveis com Graphify, contra 81% (Sem), 85% (baseline), 83% (auto+otimizador) e 82% (JEV), conforme medido pela expressão regular da resposta nos contextos armazenados (uma aproximação). A classe duplicatas/versões (3 perguntas) é a única em que auto+otimizador fica muito abaixo de Sem (41% contra 67%), coerente com a observação do benchmark de recuperação de que algumas notas `-pre-` substituídas ainda são entregues; a amostra é anedótica.

Tabela V
ACURÁCIA (%) POR CLASSE DE PERGUNTA, NOVE MODELOS AGREGADOS. CLASSES COM $n \leq 3$ PERGUNTAS SÃO ANEDÓTICAS.

Classe	n	Sem	Base.	Graph.	Auto	G+JEV
factual	36	87	94	85	91	89
numérica	28	94	91	57	91	87
duplicatas/versões	3	67	48	67	41	67
ambígua	3	63	70	70	67	70
documento longo	2	39	56	50	56	50

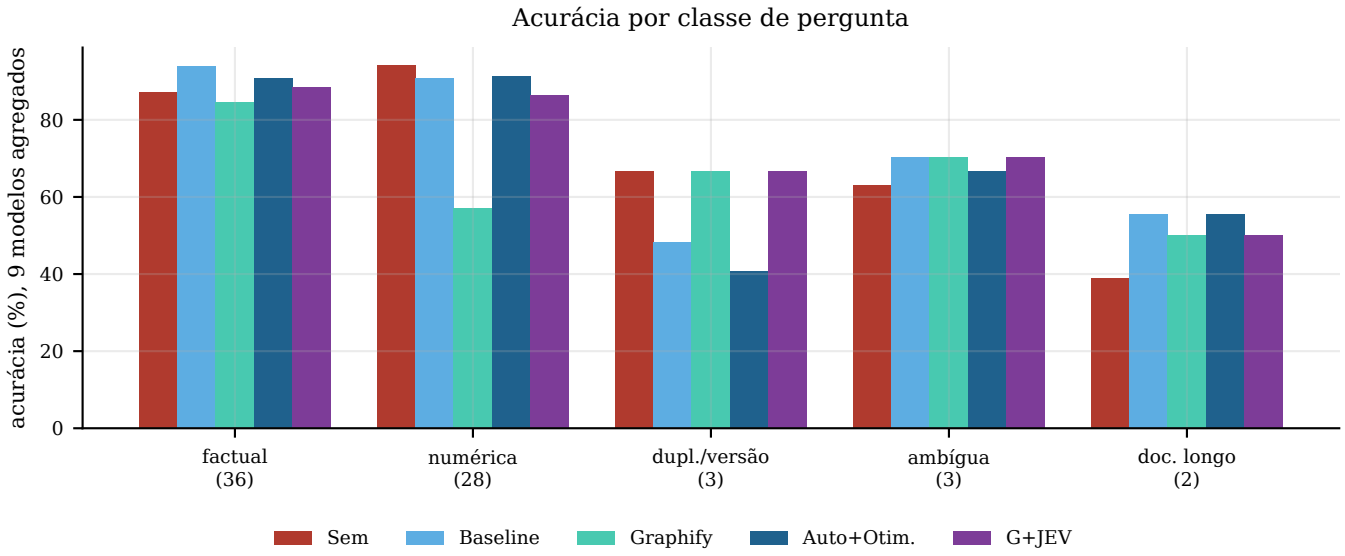


Figura 14. Acurácia por classe de pergunta (nove modelos agregados).

Tabela VI

ACURÁCIA ESTRATIFICADA (%), NOVE MODELOS AGREGADOS. *Com recuperação* = AS 60 PERGUNTAS RESPONDÍVEIS PARA AS QUAIS O APARATO *Sem* ENTREGOU UM CONTEXTO; *Sem-vazio* = AS 12 PERGUNTAS RESPONDÍVEIS NAS QUAIS SEU MECANISMO θ SE ABSTEVE (CONTEXTO VAZIO). BLOCO SUPERIOR: VEREDITOS EXATAMENTE COMO PRODUZIDOS PELO AVALIADOR LLM. BLOCO INFERIOR: REGRA DE SENSIBILIDADE R1 (UMA RESPOSTA QUE COMEÇA COM A FRASE DE ABSTENÇÃO EXIGIDA NUNCA É PONTUADA COMO CORRETA). Δ = DIFERENÇA PAREADA EM RELAÇÃO A *Sem* EM PP, COM IC BOOTSTRAP DE 95%.

Pontuação	Estrato (n)	Sem	Baseline	Graphify	Auto+otim.	G+JEV
Avaliador original	todas respondíveis (72)	86.6	88.7	71.6	86.9	85.0
		Δ	+2.2 [−4.9,+9.4]	−15.0 [−25.5,−4.2]	+0.3 [−7.4,+8.0]	−1.5 [−10.6,+7.9]
Avaliador original	com recuperação (60)	94.3	89.3	68.0	87.4	83.0
		Δ	−5.0 [−10.0,−0.7]	−26.3 [−36.5,−17.0]	−6.9 [−12.4,−1.9]	−11.3 [−18.9,−4.6]
Avaliador original	Sem-vazio (12)	48.1	86.1	89.8	84.3	95.4
		Δ	+38.0 [+10.2,+67.6]	+41.7 [+16.7,+66.7]	+36.1 [+6.5,+64.8]	+47.2 [+16.7,+75.9]
Regra R1	todas respondíveis (72)	76.9	84.0	55.9	81.6	80.2
		Δ	+7.1 [−2.6,+16.5]	−21.0 [−35.8,−6.0]	+4.8 [−5.6,+15.3]	+3.4 [−9.4,+15.7]
Regra R1	com recuperação (60)	92.2	85.6	50.6	83.3	77.8
		Δ	−6.7 [−14.3,−0.4]	−41.7 [−53.7,−29.3]	−8.9 [−17.0,−1.7]	−14.4 [−23.7,−5.6]
Regra R1	Sem-vazio (12)	0.0	75.9	82.4	73.1	92.6
		Δ	+75.9 [+54.6,+94.4]	+82.4 [+62.0,+98.1]	+73.1 [+49.1,+92.6]	+92.6 [+77.8,+100.0]

C. Análise estratificada e um defeito de pontuação

O aparato *Sem* se abstém quando sua maior pontuação BM25 fica abaixo de θ, portanto forneceu contexto vazio para 12 das 72 perguntas respondíveis. A divisão por esse evento (Tabela VI) revela que a diferença agregada próxima de zero é a soma de dois efeitos opostos. Nas 60 perguntas em que *Sem* recuperou algo, ele obteve 94.3%, contra 87.4% de auto+otimizador (-6.9 pp pareados, IC $[-12.4, -1.9]$), 89.3% de baseline (-5.0, IC $[-10.0, -0.7]$) e 83.0% de JEV (-11.3). Nas 12 perguntas com contexto *Sem* vazio, Anamnésia foi muito melhor (+36 a +47 pp).

A pontuação de *Sem* nessas 12 perguntas (48.1%) é implausivelmente alta para uma resposta que deveria ser uma abstenção. A inspeção das respostas armazenadas mostrou que o avaliador rotulou como CORRECT algumas respostas consistindo apenas na frase de abstenção exigida, apesar das instruções. Como isso é um defeito da nossa avaliação, relatamos uma regra determinística de sensibilidade R1 (uma resposta que começa com a frase de abstenção nunca é CORRECT em uma pergunta respondível), aplicada ao texto armazenado; ela altera 63 vereditos de *Sem* e entre 31 e 102 por braço de Anamnésia. Sob R1, a acurácia agregada passa a 76.9% (Sem), 84.0% (baseline), 55.9% (Graphify), 81.6% (auto+otimizador) e 80.2% (JEV); as diferenças pareadas agregadas ($+4.8$ para auto+otimizador, IC $[-5.6, +15.3]$) continuam estatisticamente indistinguíveis de zero, mas o padrão estratificado persiste (-8.9 pp nas 60 perguntas com recuperação, IC $[-17.0, -1.7]$). Portanto, consideramos a seguinte leitura a mais bem sustentada: *Anamnésia reduz os tokens em cerca de 84% e é mais robusto do que um BM25 de notas completas com limiar quando esse BM25 não recupera nada, mas, quando o BM25 de notas completas recupera algo, fornecer ao modelo notas inteiras produz acurácia mensuravelmente maior do que o contexto recortado*. O resultado principal "86.9% contra 86.6%" é exato como valor avaliado na execução original, mas não deve ser interpretado como "sem custo de qualidade".

D. Perguntas sem resposta

Todos os nove modelos, em todos os cinco braços, abstiveram-se corretamente em todas as dez perguntas sem resposta, conforme a avaliação. Isso reflete o seguimento de instruções pelo gerador, não o gateway: o benchmark de recuperação mostra que Anamnésia *sempre* retorna um contexto (taxa de abstenção de 0% no conjunto sem resposta, taxa de falsos positivos de 100%), enquanto o aparato *Sem* com limiar se abstém corretamente em 100% das vezes, mas também se abstém incorretamente em 16.7% das perguntas respondíveis.

E. Evidências apenas de recuperação (24 de outubro, medições reais)

A Tabela VII e a Fig. 15 resumem D2. Em relação a *Sem* (OFF-A), os pipelines de Anamnésia entregam 77–86% menos tokens de contexto, com maior recall@10 (0.97 contra 0.83); a taxa de presença da resposta de referência no contexto de Graphify (0.56) fica muito abaixo de baseline (0.83), correspondendo à perda generativa. Em relação ao OFF-B mais forte (BM25 por seções com o mesmo orçamento), Anamnésia entrega 8–10% *mais* tokens (IC inclui zero), mas maior recall@10 ($\Delta = +0.21$ a $+0.24$ pareado, ICs excluindo zero) e maior presença da resposta no contexto ($+0.10$ baseline; $+0.135$ auto, IC $[0.02, 0.25]$). Assim, diante de uma referência competente com segmentação, a vantagem está na qualidade da recuperação, não no custo de tokens; o benchmark generativo não incluiu OFF-B. Latência (Fig. 16): auto+otimizador tem p50 de 45.6 ms a frio e 9.9 ms a quente (cache de resultados), contra 0.7 ms da referência em processo, isto é, dezenas de vezes mais lento, porém desprezível em relação à latência de segundos dos LLMs; Laya (CPU local) precisa de ≈ 4 s por consulta e entregou 6.7k tokens (redução de 40%) com recall@10 de 0.75.

F. Cenários de persistência, atualização e isolamento (24 de outubro)

Pequenos cenários controlados exercitaram os mecanismos de memória. (i) *Gravação e busca com escopo* (F9-01): 18/18 verificações passaram para os três pipelines de Anamnésia (uma nota salva em um projeto é encontrada no seu escopo, mas não em outro). (ii) *Isolamento* no corpus sintético: sem escopo, 68–82% das fontes entregues vinham de outros projetos; com `projeto:X`, 0% de vazamento com hit@10 ≥ 0.97. (iii) *Atualização com o servidor em execução*: 3 s após editar uma nota, a versão antiga ainda era servida; aos 7 s, a nova versão era servida por baseline e auto, mas o braço Graphify não refletiu a edição em nenhum dos momentos (o grafo é construído por sessão). (iv) *Injeção/desinformação* (N=5 e 3): a nota-isca foi entregue por todos os pipelines; o otimizador sinalizou injeção em 5/5 e desinformação em 2/3 casos, sem bloquear. (v) D3 (antes/depois de uma alteração

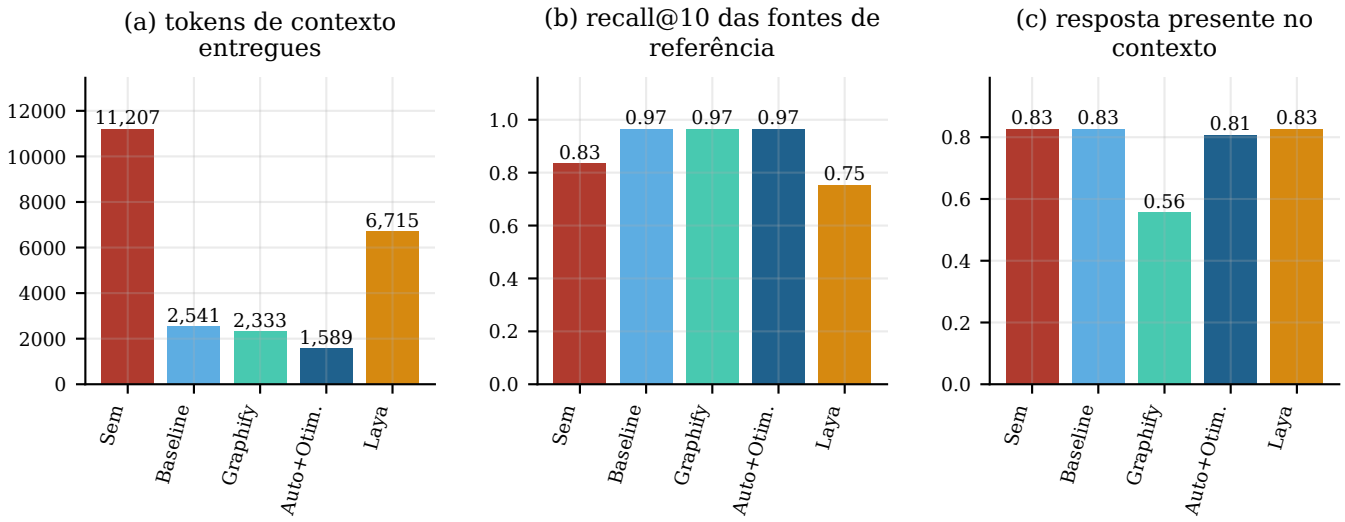


Figura 15. Benchmark apenas de recuperação em D2 (partição de teste): tokens de contexto, recall@10 e presença da resposta no contexto entregue.

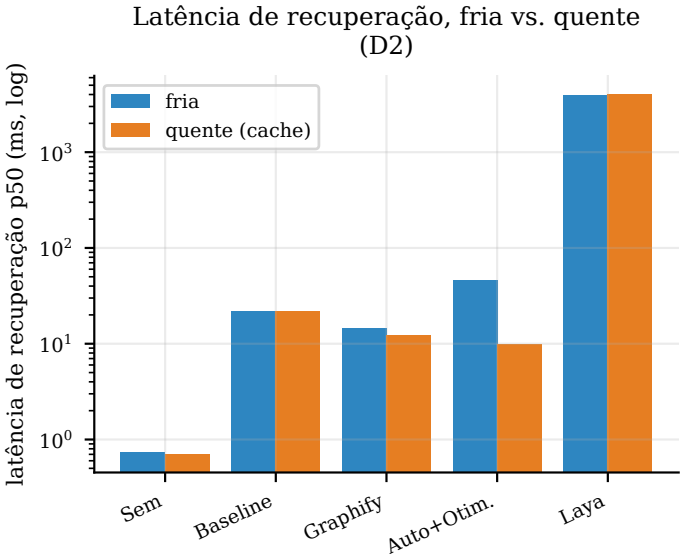


Figura 16. Latência de recuperação, a frio contra a quente.

Tabela VII
BENCHMARK APENAS DE RECUPERAÇÃO DE 24 DE OUTUBRO DE 2026 (D2, PARTIÇÃO DE TESTE, 56 CONSULTAS/52 RESPONDÍVEIS, 5 REPETIÇÕES MEDIDAS, TIKTOKEN `cl100k_base`). A LATÊNCIA É O P50 DA IDA E VOLTA DO CLIENTE (MCP STDIO PARA *Com*; EM PROCESSO PARA *Sem*).

Braço	Tok. ctx	R@5	R@10	Resp. no ctx	Frio ms	Quente ms
Sem (OFF-A)	11,207	0.77	0.83	0.83	0.7	0.7
BM25 por seções @ orçamento auto (OFF-B)	1,437	0.67	0.73	0.67	5.6	4.3
Baseline	2,541	0.89	0.97	0.83	21.9	21.6
Graphify híbrido	2,333	0.89	0.97	0.56	14.7	12.3
Auto + otimizador	1,589	0.85	0.97	0.81	45.6	9.9
Laya (local, CPU)	6,715	0.73	0.75	0.83	3970.9	4062.6

estrutural do cofre, 67 perguntas pareadas): recall@10 e presença da resposta no contexto inalterados ($\Delta = 0$), mudanças de recall@5 de -0.5 a -3.5 pp com ICs tocando zero. Estas são *verificações de mecanismos com N pequeno*; não demonstram que um agente se lembra de conversas anteriores.

VIII. DISCUSSÃO

O que foi demonstrado. (1) O gateway reduz de forma confiável o prompt em aproximadamente 84% em relação a BM25 de notas completas e, com os preços do próprio modelo, converte isso em economia de custos de 80–86% para o pipeline padrão em oito modelos remotos (custo de geração medido; zero dólares de recuperação). (2) A acurácia agregada conforme avaliada não é distinguível da referência de notas completas, tanto para o pipeline padrão quanto para a recuperação baseline simples. (3) A qualidade da recuperação do pipeline padrão supera BM25 de notas completas e de seções quanto à recuperação das fontes de referência.

O que não foi mostrado, ou foi contradito. (a) O avaliador pago não compensou seu custo aqui: acrescentou US$0.08 em oito execuções e ampliou os contextos (2,908 contra 1,776 tokens), sem ganho de acurácia (85.0% contra 86.9%). Uma razão plausível é que o pipeline JEV parte do conjunto de candidatos de Graphify, cuja recuperação de seções contendo respostas é baixa; não isolamos isso. (b) O grafo de Graphify é a ferramenta errada para *encontrar* texto: contém rótulos, não corpos, e as pontuações em escada de posições interagem mal com o corte adaptativo de 0.5. O híbrido corrige a recuperação de *arquivos* (recall@10 0.97), mas não de *trechos contendo respostas* (0.56). (c) A paridade do resultado principal esconde um déficit estratificado em relação a BM25 de notas completas quando este recupera algo (Seção VII-C).

Por que o contexto recortado pode perder. Um corte adaptativo e um limite de 300 tokens por trecho podem remover a frase exata que contém um número; notas completas a incluem. A perda nas perguntas numéricas para Graphify (57%) sustenta essa interpretação; para baseline e auto, ela é de poucos pontos. Ajustar os parâmetros de trecho e orçamento em uma partição de desenvolvimento e acrescentar alternativas de notas completas no nível de seção são soluções naturais que não tentamos.

Onde a economia importa. Economias absolutas são relevantes apenas para modelos cobrados e contextos longos; para modelos baratos, o custo adicional de JEV consome até um terço da economia (mistral: 83% $\rightarrow$ 59%). Para modelos locais, o benefício está na latência e na pressão sobre a janela de contexto, não em dólares.

IX. LIMITAÇÕES E AMEAÇAS À VALIDADE

Validade interna. Uma repetição por célula; ICs de 95% por célula são de cerca de ± 8 pp. O avaliador é um único LLM, não foi validado contra humanos e apresentou o defeito de pontuação de abstenções da Seção VII-C; R1 é uma correção heurística, não uma nova avaliação. O mecanismo de contexto vazio (θ) do braço *Sem* é, por si só, uma escolha de projeto que interage com a acurácia. As expressões regulares de referência foram escritas para contextos, não para respostas livres, e são usadas apenas na análise de presença no contexto. Avaliador, gerador e alguns modelos avaliados pertencem às mesmas famílias de fornecedores, portanto não se pode excluir autopreferência.

Força da referência. *Sem* entrega notas completas (média de 10.9k tokens, máximo de 71k), o que é uma referência fraca para custo; um BM25 por seções (OFF-B) foi avaliado apenas para recuperação, não para geração. Parte da economia de tokens é atribuível à segmentação, que não é exclusiva de Anamnésia.

Validade de construto. Custo significa custo de tokens; energia, tempo de engenharia e custo de manutenção do gateway são excluídos. O custo JEV é calculado a partir de tarifa publicada e contagens de tokens, não de fatura. Tokenizadores de provedores diferem, portanto "tokens de entrada" não são comparáveis entre modelos em termos absolutos.

Validade externa. Um único cofre de engenharia em português, de um único autor (379 notas); 82 perguntas; um retrato dos modelos em 2026. Resultados podem não se transferir para outros idiomas, corpora ou para um cofre com muitas paráfrases, no qual a recuperação lexical seja fraca.

Não avaliado. Persistência entre sessões reais de agentes, recuperação após conversas longas com múltiplos turnos, consolidação de memória, sucesso em tarefas de ponta a ponta, comparação com sistemas densos ou graph-RAG, Decision Engine, Laya na geração, estratégias híbridas/ensemble sem rota de recuperação, modelos premium e provedores cujas chaves existem, mas que não foram testados. Afirmações presentes no README do projeto (por exemplo, "-39.7%/-24.2% de contexto com recall inalterado" e comparações de tokens de esquema) não têm artefato armazenado no material auditado, e o benchmark de 24 de outubro mostra recall@5 em D1 caindo de 0.903 para 0.799 no braço do otimizador, enquanto recall@10 permanece inalterado.

Segurança. A triagem de injeção é heurística e não bloqueante; notas recuperadas são rotuladas como não confiáveis, mas o agente consumidor decide o que fazer com elas. Endpoints de escrita são protegidos por loopback e token; leituras em loopback não exigem token.

X. Trabalhos Futuros

1) **Avaliação de memória de longo horizonte**: executar tarefas no estilo LoCoMo e LongMemEval [19], [20] com agentes reais entre sessões, medindo a recuperação de decisões após dezenas de turnos com e sem `memory_save`.
2) **Referências generativas justas**: acrescentar OFF-B (BM25 por seções com orçamento igual) e um recuperador denso ao estudo generativo; acrescentar repetições e avaliação validada por humanos.
3) **Ajuste de parâmetros**: limite de trechos, razão do corte adaptativo e orçamento ajustados em uma partição de desenvolvimento; alternativas no nível de seção para perguntas numéricas.
4) **Um grafo autossuficiente**: gerar o grafo de links/títulos dentro de Anamnésia (wikilinks, títulos, frontmatter) e medir recall em relação ao grafo de Graphify, removendo a dependência externa da construção do grafo.
5) **Economia do avaliador**: avaliar JEV em um conjunto de candidatos com maior recall (baseline em vez de sementes Graphify), com custo faturado, não calculado.
6) **Atualização de grafos**: atualização incremental do grafo para que edições se tornem visíveis dentro do intervalo de atualização do cofre.
7) **Defesas mais fortes contra injeção**: políticas de bloqueio e detectores aprendidos, avaliados em um conjunto adversarial maior.

XI. Conclusão

Anamnésia é um gateway de memória funcional, local e independente de agente, que implementa recuperação lexical, recuperação enriquecida por grafo, recorte de contexto, remoção de quase duplicatas, cache, sinalização de injeção e registro de custos por trás de interfaces MCP e REST. Em um benchmark real, integralmente registrado, seu pipeline padrão reduziu os tokens de entrada em cerca de 84% e o custo efetivo em cerca de 82% em oito modelos remotos, com acurácia agregada estatisticamente indistinguível de uma referência BM25 de notas completas; porém, uma reanálise estratificada mostra menor acurácia quando essa referência efetivamente recuperou algo, o pipeline Graphify prejudicou a acurácia em 15 pp e o avaliador pago JEV não teve boa relação custo-benefício nesse cenário. Se o projeto de memória persistente melhora a continuidade de agentes em sessões reais e longas continua sendo uma questão empírica em aberto que este artigo deliberadamente não responde.

DISPONIBILIDADE

O pacote público `assets/paper-v2.1.0/` contém os fontes LaTeX, os PDFs em inglês e português, as figuras bilíngues, os dados agregados das figuras e os scripts de visualização. Os logs originais de geração e avaliação (`calls.jsonl`, `judge.jsonl`), o instantâneo privado do cofre e as entradas originais da avaliação permanecem locais em `data/reports-out/`; não estão incluídos no pacote público. Os agregados publicados reproduzem as figuras, não o experimento generativo completo. O artigo não acrescenta novos experimentos com modelos nem novos gastos com APIs.

AGRADECIMENTOS

Agradecemos aos mantenedores de Graphify, SQLite, FastAPI, Model Context Protocol, Ollama, OpenRouter e TypeSafe AI, sobre cujo trabalho Anamnésia se apoia.

REFERÊNCIAS

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv:2005.11401. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023. [Online]. Available: https://arxiv.org/abs/2312.10997

[4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[5] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th ACM UIST*, 2023.

[6] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in *Proc. AAAI*, 2024, arXiv:2305.10250. [Online]. Available: https://arxiv.org/abs/2305.10250

[7] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," *arXiv preprint arXiv:2504.19413*, 2025. [Online]. Available: https://arxiv.org/abs/2504.19413

[8] SQLite Consortium, "SQLite FTS5 extension (including the `bm25()` auxiliary function)," https://www.sqlite.org/fts5.html, 2026, accessed 25 Oct 2026.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[10] Graphify Labs, "Graphify: turn a folder of code and documents into a queryable knowledge graph (pypi package `graphifyy` 0.9.59, apache-2.0)," https://github.com/Graphify-Labs/graphify, 2026, accessed 25 Oct 2026.

[11] TypeSafe AI, "TypeSafe api and models documentation (jev, `jev-1.13.0`; python sdk `typesafe-sdk` 0.7.1)," https://docs.typesafe.ai, 2026, accessed 25 Oct 2026.

[12] Model Context Protocol contributors, "Model context protocol specification (2025-06-18)," https://modelcontextprotocol.io/specification/2025-06-18, 2025, accessed 25 Oct 2026.

[13] S. Ramírez, "FastAPI framework documentation," https://fastapi.tiangolo.com, 2026, accessed 25 Oct 2026.

[14] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020, arXiv:2004.04906. [Online]. Available: https://arxiv.org/abs/2004.04906

[15] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd ACM SIGIR*, 2009.

[16] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[17] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2405.14831. [Online]. Available: https://arxiv.org/abs/2405.14831

[18] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999. [Online]. Available: http://ilpubs.stanford.edu:8090/422/

[19] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. ACL*, 2024, arXiv:2402.17753. [Online]. Available: https://arxiv.org/abs/2402.17753

[20] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. ICLR*, 2025, arXiv:2410.10813. [Online]. Available: https://arxiv.org/abs/2410.10813

[21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023.

[22] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression," *arXiv preprint arXiv:2310.06839*, 2023. [Online]. Available: https://arxiv.org/abs/2310.06839

[23] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023. [Online]. Available: https://arxiv.org/abs/2305.05176
[24] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," *arXiv preprint arXiv:2406.18665*, 2024. [Online]. Available: https://arxiv.org/abs/2406.18665
[25] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th ACM STOC*, 2002.
[26] G. S. Manku, A. Jain, and A. Das Sarma, "Detecting near-duplicates for web crawling," in *Proc. 16th WWW*, 2007.
[27] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997.
[28] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2023, arXiv:2306.05685. [Online]. Available: https://arxiv.org/abs/2306.05685
[29] Ollama contributors, "Ollama: run large language models locally," https://github.com/ollama/ollama, 2026, accessed 25 Oct 2026.
[30] Convai Innovations, "Laya: a fast, non-autoregressive system 1 decision engine (pypi `laya`, apache-2.0)," https://huggingface.co/convaiinnovations/laya, 2026, accessed 25 Oct 2026.
[31] Pydantic Services Inc., "Pydantic documentation," https://docs.pydantic.dev, 2026, accessed 25 Oct 2026.
[32] OpenAI, "tiktoken: a fast BPE tokeniser," https://github.com/openai/tiktoken, 2026, accessed 25 Oct 2026.
[33] OpenRouter, Inc., "OpenRouter api documentation," https://openrouter.ai/docs, 2026, accessed 25 Oct 2026.
[34] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
[35] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.